$\mathcal{T}_{\mathcal{A}'} \circ \mathcal{T}_{\phi^{-1}}$ then computes, for $\boldsymbol{y} \in \Sigma^*, \boldsymbol{x}' \in \Delta^*$,

$$\begin{aligned}
\left(\mathcal{T}_{\mathcal{A}'} \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}, \boldsymbol{x}') &= \sum_{\boldsymbol{x} \in \Delta^*} \mathcal{T}_{\phi^{-1}}(\boldsymbol{y}, \boldsymbol{x}) \cdot \mathcal{T}_{\mathcal{A}'}(\boldsymbol{x}, \boldsymbol{x}') && \text{(50a, Eq. 15.4 in Pereira and Riley (1997).)} \\
&= \sum_{\boldsymbol{x} \in \Delta^*} \mathbb{1}\left\{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})\right\} \cdot \mathcal{T}_{\mathcal{A}'}(\boldsymbol{x}, \boldsymbol{x}') && \text{(50b)} \\
&= \sum_{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})} \mathcal{T}_{\mathcal{A}'}(\boldsymbol{x}, \boldsymbol{x}') && \text{(50c)} \\
&= \sum_{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})} \mathbb{1}\left\{\boldsymbol{x} = \boldsymbol{x}'\right\} \mathcal{A}(\boldsymbol{x}) && \text{(50d)} \\
&= \mathbb{1}\left\{\boldsymbol{x}' \in \phi^{-1}(\boldsymbol{y})\right\} \mathcal{A}(\boldsymbol{x}') && \text{(50e)}
\end{aligned}$$

We then turn $\mathcal{T}_{\mathcal{A}'} \circ \mathcal{T}_{\phi^{-1}}$ into a PFSA in the standard way by projecting the WFST onto the transition input labels (Mohri et al., 2008). This results in a PFSA that assigns the string $\boldsymbol{y} \in \Sigma^*$ the probability that equals the sum of all possible mappings of $\boldsymbol{y}$ to any $\boldsymbol{x}' \in \Delta^*$:

$$\begin{aligned}
\left(\mathcal{T}_{\mathcal{A}'} \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}) &\stackrel{\text{def}}{=} \sum_{\boldsymbol{x}' \in \Delta^*} \left(\mathcal{T}_{\mathcal{A}'} \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}, \boldsymbol{x}') && \text{(51a)} \\
&= \sum_{\boldsymbol{x}' \in \Delta^*} \mathbb{1}\left\{\boldsymbol{x}' \in \phi^{-1}(\boldsymbol{y})\right\} \mathcal{A}(\boldsymbol{x}') && \text{(51b)} \\
&= \sum_{\boldsymbol{x}' \in \phi^{-1}(\boldsymbol{y})} \mathcal{A}(\boldsymbol{x}') && \text{(51c)} \\
&= \mathcal{A}'\left(\phi^{-1}(\boldsymbol{y})\right) && \text{(51d)} \\
&= \left(\mathcal{A}' \circ \phi^{-1}\right)(\boldsymbol{y}) && \text{(51e)} \\
&= \mathcal{A}(\boldsymbol{y}) && \text{(51f)}
\end{aligned}$$

This shows that $\mathcal{A}' \circ \phi^{-1}$ is weakly equivalent to $\mathcal{A}$, which finishes the proof. ■

E.2 Nondeterminism in PPDAs

Theorem E.1. *Let $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, (q_\iota, \mathrm{S}), (q_\varphi, \varepsilon))$ be a PPDA. Then, there exists a* deterministic *PPDA $\mathcal{P}'$ over the alphabet $(\Sigma_\varepsilon) \times Q \times \Gamma$ with the state space $(\Sigma_\varepsilon) \times Q \times \Gamma$ that is regularly reducible to $\mathcal{P}$.*

Proof. Given a PPDA $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, (q_\iota, \mathrm{S}), (q_\varphi, \varepsilon))$, we prove the existence of a *deterministic* PPDA $\mathcal{P}'$ over the alphabet $\Sigma_\varepsilon \times Q \times \Gamma$ that is regularly reducible to $\mathcal{P}$.

We construct the deterministic PPDA $\mathcal{P}' = \left(\Sigma_\varepsilon \times Q \times \Gamma, \Sigma_\varepsilon \times Q \times \Gamma, \Gamma, \delta', (q_\iota, \mathrm{S}), (q_\varphi, \varepsilon)\right)$ with the set of transitions

$$\begin{aligned}
\delta' \stackrel{\text{def}}{=} \{(y, q, \gamma), \gamma' \xrightarrow{(y', q', \gamma')/w} (y', q', \gamma'), \boldsymbol{\gamma} \mid q, \gamma' \xrightarrow{y'/w} q', \boldsymbol{\gamma} \in \delta, \\
y \in \Sigma_\varepsilon, \gamma \in \Gamma\}. \qquad (52)
\end{aligned}$$

- **Probabilistic:** For any $(y, q, \gamma) \in \Sigma_\varepsilon \times Q \times \Gamma$ and $\gamma' \in \Gamma$,

$$\sum_{(y,q,\gamma),\gamma' \xrightarrow{(y',q',\gamma')/w} (y',q',\gamma'),\boldsymbol{\gamma}} w = \sum_{q,\gamma' \xrightarrow{y'/w} q',\boldsymbol{\gamma}} w \qquad \text{(53a)}$$

$$= 1 \qquad \text{(53b)}$$

- **Deterministic:** Just like in the finite-state case, let $(y, q, \gamma) \in \Sigma_\varepsilon \times Q \times \Gamma$ be a state, $\gamma' \in \Gamma$ a stack symbol and $(y', q', \gamma') \in \Sigma_\varepsilon \times Q \times \Gamma$ an input symbol. Then (y', q', γ') uniquely determines the next configuration.

We now show that $\mathcal{P}'$ is regularly reducible to $\mathcal{P}$. More precisely, we define the regular function $\phi(y, q, \gamma) \stackrel{\text{def}}{=} y$ and show that $\mathcal{P}' \circ \phi^{-1}$ is weakly equivalent to $\mathcal{P}$.

Let $\mathcal{G}'$ be a CFG[26] that is equivalent to $\mathcal{P}'$ (see Hopcroft et al., 2001, Section 6.3.2 for the construction). Using grammars, rather than pushdown automata, will simplify the proof as there are well-known constructions (e.g., Bar-Hillel et al. (1961), Pasti et al. (2023)) for composing CFGs and FSTs. We therefore prove that $\mathcal{G}' \circ \phi^{-1}$ is weakly equivalent to $\mathcal{P}$.

Recall that ϕ^{-1} can be implemented as an FST $\mathcal{T}_{\phi^{-1}}$ defined as

$$\mathcal{T}_{\phi^{-1}}(\boldsymbol{y}, \boldsymbol{x}) \stackrel{\text{def}}{=} \mathbb{1}\left\{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})\right\}. \tag{54}$$

Adapting Corollary 1 from Pasti et al. (2023) to the FST case, we get

$$\begin{aligned}\left(\mathcal{G}' \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}, \boldsymbol{x}) &= \mathcal{G}'(\boldsymbol{x}) \cdot \mathcal{T}_{\phi^{-1}}(\boldsymbol{y}, \boldsymbol{x}) &\text{(55a)}\\ &= \mathcal{G}'(\boldsymbol{x}) \cdot \mathbb{1}\left\{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})\right\}. &\text{(55b)}\end{aligned}$$

Then, just like in the regular case, we compute $\left(\mathcal{G}' \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y})$ by summing over all possible $\boldsymbol{x} \in \Sigma^*$ values:

$$\begin{aligned}\left(\mathcal{G}' \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}) &\stackrel{\text{def}}{=} \sum_{\boldsymbol{x} \in \Sigma^*} \left(\mathcal{G}' \circ \mathcal{T}_{\phi^{-1}}\right)(\boldsymbol{y}, \boldsymbol{x}) &\text{(56a)}\\ &= \sum_{\boldsymbol{x} \in \Sigma^*} \mathcal{G}'(\boldsymbol{x})\, \mathbb{1}\left\{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})\right\} &\text{(56b)}\\ &= \sum_{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})} \mathcal{G}'(\boldsymbol{x}) &\text{(56c)}\\ &= \sum_{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})} \mathcal{P}'(\boldsymbol{x}) &\text{(56d)}\\ &= \mathcal{P}'\left(\phi^{-1}(\boldsymbol{y})\right) &\text{(56e)}\\ &= \left(\mathcal{P}' \circ \phi^{-1}\right)(\boldsymbol{y}) &\text{(56f)}\\ &= \mathcal{P}(\boldsymbol{y}). &\text{(56g)}\end{aligned}$$

This shows that $\mathcal{G}' \circ \mathcal{T}_{\phi^{-1}}$ is weakly equivalent to $\mathcal{P}$, thus $\mathcal{P}' \circ \mathcal{T}_{\phi^{-1}}$ is weakly equivalent to $\mathcal{P}$. This concludes the proof. ■

F Proofs: Representational Capacity of Neural LMs

F.1 Finite-state Language Models

Theorem 4.1. *For any regular LM, there exists a weakly equivalent CoT-augmented constant-precision Elman RNN LM.*

Proof. Let $\mathcal{A} = (\Sigma, Q, \lambda, \rho, \delta)$ be a PFSA. We divide the definition of a weakly equivalent Elman LM $\mathcal{R} = (\Sigma \times Q, D, \mathbf{U}, \mathbf{V}, \mathbf{b}, \boldsymbol{\eta})$ with the output matrix $\mathbf{E}$ into multiple steps.[27]

Note: The paper assumes a BOS-padding of the input strings. Since CoT-augmented LMs work over a potentially larger alphabet, the padding symbol has to be changed accordingly. Particularly, the CoT-augmented RNN will work over the alphabet $\Sigma \times Q$. For simplicity, assume that any input string is padded by the symbol (BOS, q_0) for some arbitrary (but fixed) state $q_0 \in Q$.

[26] Similar to the notation used for PPDAs, we use $\mathcal{G}(\boldsymbol{y})$ to denote the weight of the string $\boldsymbol{y}$ under the grammar $\mathcal{G}$.

[27] Notice that $\mathcal{R}$ is CoT-augmented by construction, as it works over the alphabet $\Delta \stackrel{\text{def}}{=} \Sigma \times Q$. Δ also satisfies the Σ-augmentation condition.

Hidden states. Let $D = |\underline{\Sigma}_\varepsilon||Q|$. We will represent the input symbols with the one-hot representation function $[\![\cdot,\cdot]\!]$ that computes $[\![\underline{y}, q]\!] \in \{0,1\}^{|\underline{\Sigma}_\varepsilon||Q|}$:[28]

$$[\![\underline{y}, q]\!]_{\underline{y},q} \stackrel{\text{def}}{=} 1 \tag{57}$$

while other entries of the vector are zero.[29] We can define $\boldsymbol{\eta} \stackrel{\text{def}}{=} \mathbf{0}_D$.

Recurrence. Conveniently, the CoT-augmented RNN will store the current PFSA state in its output "symbol". Therefore, its next hidden state $\mathbf{h}_t$ will not, in fact, depend on the previous one ($\mathbf{h}_{t-1}$); $\mathbf{h}_{t-1}$ will only be used to compute the next-symbol–state output distribution through Eq. (2). The RNN therefore only has to appropriately incorporate the input information. With this in mind, we define the following Elman RNN parameters $\mathbf{U}, \mathbf{V}$, and $\mathbf{b}$:

$$\mathbf{U} \stackrel{\text{def}}{=} \mathbf{O}_D \qquad \mathbf{U} \in \mathbb{R}^{D\times D} \tag{58a}$$

$$\mathbf{V} \stackrel{\text{def}}{=} \mathbf{I}_D \qquad \mathbf{V} \in \mathbb{R}^{D\times D} \tag{58b}$$

$$\mathbf{b} \stackrel{\text{def}}{=} \mathbf{0}_D \qquad \mathbf{b} \in \mathbb{R}^{D} \tag{58c}$$

where $\mathbf{O}_D$ is a $D \times D$-dimensional matrix of zeros, $\mathbf{I}_D$ is the D-dimensional identity matrix, and $\mathbf{0}_D$ is a D-dimensional vector of zeros. Then, we define the output matrix $\mathbf{E} \in \mathbb{R}^{|\underline{\Sigma}_\varepsilon||Q|\times D}$ as

$$E_{(y',q'),(y,q)} \stackrel{\text{def}}{=} p\left(q', y' \mid q\right) \qquad q, q' \in Q, \quad y \in \Sigma_\varepsilon, y' \in \Sigma_\varepsilon \tag{59a}$$

$$E_{(\varepsilon,q),(\text{BOS},q_0)} \stackrel{\text{def}}{=} \lambda\left(q\right) \qquad q \in Q \tag{59b}$$

$$E_{(\text{EOS},q),(y,q)} \stackrel{\text{def}}{=} \rho\left(q\right) \qquad q \in Q, \quad y \in \Sigma_\varepsilon. \tag{59c}$$

Computation of string probabilities. We now show that $\mathcal{R}$ with $\mathbf{E}$ defines the same conditional distributions as $\mathcal{A}$, meaning that it implicitly defines the same probability distribution over strings. As mentioned above, we assume that the input string is prefixed with the (BOS, q_0) symbol. Let the previously generated output symbol $\in \Delta$ be $\left(\underline{y}, q\right)$ (in the first step, the "generated" pair is (BOS, q_0)). We get that

$$\mathbf{h}_t = \mathrm{H}\left(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{V}[\![\underline{y}, q]\!] + \mathbf{b}\right) \tag{60a}$$

$$= \mathrm{H}\left(\mathbf{O}_D\mathbf{h}_{t-1} + \mathbf{V}[\![\underline{y}, q]\!] + \mathbf{0}_D\right) \tag{60b}$$

$$= \mathrm{H}\left(\mathbf{V}[\![\underline{y}, q]\!]\right) \tag{60c}$$

$$= [\![\underline{y}, q]\!] \tag{60d}$$

The one-hot encoding $[\![\underline{y}, q]\!]$ is then used to "index" the appropriate column of the output matrix $\mathbf{E}$ as $\mathbf{E}\mathbf{h}_t$, which contains the probabilities of the *next* symbol–state pairs defined by the PFSA, as per Eq. (59a):

$$(\mathbf{E}\mathbf{h}_t)_{(q',\overline{y}')} = E_{(\overline{y}',q'),(\underline{y},q)} = \begin{cases} \lambda\left(q'\right) & \textbf{if } \underline{y} = \text{BOS}, q = q_0, \overline{y}' = \varepsilon \\ p\left(q', \overline{y}' \mid q\right) & \textbf{otherwise} \end{cases}. \tag{61}$$

Passing $\mathbf{E}\mathbf{h}_t$ through the normalization function, the next state and symbol are sampled and passed as input to the RNN, repeating the update step. Since the Elman RNN following these dynamics by constructions generates paths (sequences of states in the output symbols) with the same probabilities as the input automaton, it enumerates all accepting paths of any string $\boldsymbol{y} \in \Sigma^*$ with the same probabilities as the original PFSA. Applying the transformation ϕ to the produced outputs and summing over sequences that yield the same string will thus result in strings sampled from the PFSA. This finishes the proof. ■

Theorem 4.2. *For any constant-precision CoT-augmented Elman RNN LM, there exists a weakly equivalent PFSA.*

[28] Throughout the paper, we index vectors and matrices directly with set elements rather than integer values, meaning each index corresponds to exactly one element from the given set.

[29] If any of the two arguments are empty, its corresponding component is also zero.